Docker ⭐ ⭐ ⭐ — Interview-Ready Q&A

1. What is Docker?
Docker is a containerization platform used to package an application along with its dependencies into a container.
A container provides a consistent environment so the application behaves similarly across development, testing, and production.
In DevOps, Docker is commonly used to build application images and deploy them consistently across environments.
Simple ga gurthupettuko:
Docker = Application + Dependencies → Container
Eng: Docker is a containerization platform that packages applications and their dependencies into portable containers.

---

2. What is a Docker Image?
A Docker image is a read-only template used to create containers.
It contains the application code, runtime, libraries, dependencies, and configuration required to run the application.
For example, we can create a Java application image containing the JDK/runtime and application JAR.
Simple ga gurthupettuko:
Image = Container create cheyyadaniki template.
Eng: A Docker image is an immutable template containing the application and its required dependencies, used to create containers.

---

3. What is a Docker Container?
A container is a running instance of a Docker image.
The image provides the packaged application and dependencies, while the container provides the runtime environment in which that image executes.
Multiple containers can be created from the same image.
Simple ga gurthupettuko:
Image = Template
Container = Running instance
Eng: A Docker container is a running instance of a Docker image.

---

4. What is the difference between Docker Image and Container?
A Docker image is a static, read-only package used as a template.
A container is a runtime instance created from that image.
For example, one application image can be used to create multiple containers.
Simple ga gurthupettuko:
Image = Blueprint
Container = Blueprint nunchi create ayina running application
Eng: An image is a packaged template, while a container is a running instance created from that image.

---

5. What is a Dockerfile?
A Dockerfile is a text file containing instructions used to build a Docker image.
It defines things such as the base image, application files, dependencies, environment variables, exposed ports, and startup command.
For example, a Dockerfile can define how to package a Java application into a container image.
Simple ga gurthupettuko:
Dockerfile = Image build cheyyadaniki instructions.
Eng: A Dockerfile contains the instructions required to build a Docker image.

---

6. What is the difference between Dockerfile and Docker Image?
A Dockerfile contains the instructions for creating an image.
The Docker image is the actual packaged result produced from those instructions.
Simple ga gurthupettuko:
Dockerfile = Recipe
Image = Recipe tho prepare ayina package
Eng: A Dockerfile defines how an image should be built, while the image is the resulting packaged artifact.

---

7. What is Docker Registry?
A Docker Registry is a storage and distribution system for Docker images.
Images can be pushed to a registry and later pulled by servers or deployment systems.
Examples include Docker Hub and private registries such as Amazon ECR.
Simple ga gurthupettuko:
Registry = Docker Images store/share cheyyadaniki place.
Eng: A Docker Registry stores and distributes container images so they can be pulled by deployment environments.

---

8. What is Docker Hub?
Docker Hub is a public registry service used to store and distribute Docker images.
Developers can pull publicly available images or push their own images depending on repository permissions.
Companies may use private registries instead when application images should not be publicly accessible.
Simple ga gurthupettuko:
Docker Hub = Popular public Docker image registry.
Eng: Docker Hub is a registry service used to store and distribute Docker images.

9. What is a Docker Tag?
A Docker tag is a label used to identify a particular image version or variant.
For example:
myapp:1.0
myapp:2.0
Tags help identify which image version should be deployed.
Simple ga gurthupettuko:
Tag = Image version/label.
Eng: A Docker tag identifies a particular version or variant of a container image.

---

10. What is docker build**?**
docker build is used to create a Docker image from a Dockerfile and its build context.
During the build, Docker processes the Dockerfile instructions and creates the image layers.
Example:
docker build -t myapp:1.0 .
Simple ga gurthupettuko:
Dockerfile → docker build → Image
Eng: docker build creates a Docker image by processing a Dockerfile and its build context.

---

11. What is docker run**?**
docker run creates and starts a container from a Docker image.
We can provide options such as port mapping, environment variables, volumes, and container names.
Example:
docker run -d -p 8080:8080 myapp:1.0
Simple ga gurthupettuko:
Image → docker run → Running Container
Eng: docker run creates and starts a container from a specified Docker image.

---

12. What is port mapping in Docker?
Port mapping connects a port on the Docker host to a port inside the container.
For example:
-p 8080:80
means host port 8080 forwards traffic to container port 80.
This allows users outside the container to access the application.
Simple ga gurthupettuko:
Host Port : Container Port
Eng: Docker port mapping forwards traffic from a host port to a port exposed by the container.

---

13. What is a Docker Volume?
A Docker volume provides persistent storage that exists independently of a container's writable layer.
This is useful for databases and applications where data must survive container deletion or recreation.
Simple ga gurthupettuko:
Volume = Container delete aina data preserve cheyyadaniki storage.
Eng: A Docker volume provides persistent storage that can survive container recreation or deletion.

---

14. Why do we need Docker Volumes?
Containers are generally considered disposable, so data stored only inside a container can be lost when the container is removed.
Volumes allow important data to be stored outside the container's writable layer.
For example, database data can be stored in a volume while the database container can be recreated safely.
Simple ga gurthupettuko:
Container temporary, Volume persistent.
Eng: Docker volumes are used to persist important data independently of the container lifecycle.

---

15. What is a Docker Network?
A Docker network provides communication between containers and controls how containers connect to each other and external networks.
For example, a web container can communicate with an application container, which can communicate with a database container through a Docker network.
Simple ga gurthupettuko:
Docker Network = Containers madhya communication.
Eng: A Docker network provides connectivity and communication between containers and other network endpoints.

---

16. What are common Docker network types?
Common Docker network drivers include **bridge, host, none, and overlay**.
The bridge network is commonly used for containers on a single Docker host.
Overlay networks are useful when container workloads span multiple Docker hosts, such as certain Docker Swarm environments.
Simple ga gurthupettuko:
Bridge = Common single-host container networking.
Overlay = Multi-host networking.
Eng: Docker provides network drivers such as bridge, host, none, and overlay for different networking requirements.

17. What is Docker Compose?
Docker Compose is a tool used to define and run multi-container applications.
It allows services, networks, volumes, environment variables, and other configuration to be described in a Compose YAML file.
For example, a web application, backend, and database can be defined together.
Simple ga gurthupettuko:
Compose = Multiple containers ni one configuration tho manage cheyyadam.
Eng: Docker Compose defines and manages multi-container applications using a YAML configuration file.

---

18. What is docker-compose.yml **/ Compose YAML?**
A Compose YAML file defines the services and configuration required for a multi-container application.
It can specify images, builds, ports, volumes, networks, environment variables, and service dependencies.
Simple ga gurthupettuko:
Compose YAML = Multi-container application configuration.
Eng: A Compose YAML file defines services and their configuration for running a multi-container application.

---

19. What is Docker Layer?
Docker images are built from multiple layers, generally corresponding to Dockerfile instructions that modify the filesystem.
Layers can be reused between images, which helps improve build efficiency and storage usage.
This is one reason Docker builds can use cache when earlier instructions have not changed.
Simple ga gurthupettuko:
Image = Multiple reusable layers.
Eng: Docker images consist of filesystem layers that can be reused and cached to improve build efficiency.

---

20. What is Docker Image Cache?
Docker can reuse previously built layers when Dockerfile instructions and their inputs have not changed.
This reduces build time because unchanged layers do not always need to be rebuilt.
In CI/CD, effective Dockerfile ordering can significantly improve build performance.
Simple ga gurthupettuko:
Same layer unchanged → Cache reuse → Faster build.
Eng: Docker build cache reuses unchanged image layers to reduce build time.

---

21. What is a multi-stage Docker build?
A multi-stage build uses multiple stages in one Dockerfile.
One stage can contain build tools and source code, while the final stage contains only the runtime requirements and application artifact.
This helps create smaller and cleaner production images.
Simple ga gurthupettuko:
Build Stage → Artifact → Small Runtime Image
Eng: A multi-stage Docker build separates the build environment from the final runtime image to reduce image size and unnecessary dependencies.

---

22. Why should Docker images be small?
Smaller images download faster, start faster, consume less storage, and reduce the attack surface.
We can reduce image size by using suitable base images, multi-stage builds, and removing unnecessary files and packages.
Simple ga gurthupettuko:
Small Image = Faster + Less Storage + Lower Attack Surface
Eng: Smaller Docker images improve deployment speed, storage efficiency, and security.

---

23. What is .dockerignore**?**
.dockerignore specifies files and directories that should not be sent as part of the Docker build context.
For example, we can exclude .git, local build files, logs, and unnecessary dependencies.
This reduces build context size and can improve build performance.
Simple ga gurthupettuko:
.dockerignore = Build context lo unnecessary files exclude cheyyadam.
Eng: .dockerignore excludes unnecessary files from the Docker build context.

---

24. What is the difference between CMD and ENTRYPOINT?
Both define what should execute when a container starts.
CMD provides default commands or arguments that can generally be overridden.
ENTRYPOINT is used to define the main executable for the container and is less easily replaced by normal command arguments.
Simple ga gurthupettuko:
CMD = Default command/arguments
ENTRYPOINT = Main executable
Eng: CMD provides default execution values, while ENTRYPOINT defines the main executable of the container.

---

25. What is the difference between COPY and ADD?
Both can copy files into a Docker image.
COPY is simpler and is generally preferred when only file copying is required.
ADD has additional behavior, including handling certain archive extraction scenarios and supporting URLs, although using explicit download steps is often clearer.

Simple ga gurthupettuko:
COPY = Simple file copy
ADD = Extra capabilities
Eng: COPY is intended for straightforward file copying, while ADD provides additional features and should be used only when those features are required.

---

26. How do you check running Docker containers?
We can use:
docker ps
This displays currently running containers along with information such as container ID, image, status, ports, and names.
To view all containers, including stopped ones, we can use:
docker ps -a
Simple ga gurthupettuko:
docker ps = Running containers
docker ps -a = All containers
Eng: docker ps lists running containers, while docker ps -a lists both running and stopped containers.

---

27. How do you check Docker container logs?
We can use:
docker logs <container>
This displays the container's standard output and error logs.
For troubleshooting, I check these logs first to identify application startup errors, configuration problems, connection failures, or runtime exceptions.
Simple ga gurthupettuko:
Container issue → docker logs → Root cause
Eng: docker logs helps troubleshoot containerized applications by displaying their standard output and error logs.

---

28. What happens if a Docker container stops?
When the main process running inside a container exits, the container normally stops.
The container itself can still exist in a stopped state until it is removed.
For troubleshooting, I check the container status and logs to understand why the main process exited.
Simple ga gurthupettuko:
Main Process Exit → Container Stop
Eng: A Docker container normally stops when its main process exits, although the stopped container can remain until removed.

---

29. How do you troubleshoot a Docker container that keeps restarting?
First, I check:
docker ps -a
Then I inspect:
docker logs <container>
I also verify the container command, environment variables, mounted volumes, dependencies, health checks, resource limits, and application configuration.
Simple ga gurthupettuko:
Restarting → Status → Logs → Command → Env → Dependencies
Eng: I troubleshoot restart loops by checking container status, logs, startup commands, configuration, dependencies, health checks, and resource constraints.

---

30. Explain a real-time Docker CI/CD flow.
A developer pushes code to GitHub, which triggers Jenkins.
Jenkins checks out the source code, builds and tests the application, and creates a Docker image.
The image is tagged with a version and pushed to a container registry such as ECR. The deployment system then pulls that image and runs it in the target environment.
Simple ga gurthupettuko:
GitHub → Jenkins → Build/Test → Docker Build → Tag → Registry → Deploy
Eng: In a typical DevOps pipeline, Jenkins builds and tests the application, creates a versioned Docker image, pushes it to a registry, and deploys that image to the target environment.